



Installing docker and running a container

Core Components of Docker

- Docker Engine acts as the heart of Docker, managing containers and images.
- Docker Images are blueprints for creating containers, encapsulating the application and its dependencies.
- Docker Containers are isolated environments created from images, running as instances of Docker applications.
- Docker Compose simplifies the configuration of applications that require multiple containers.

Install Docker Engine on Ubuntu

To get started with Docker Engine on Ubuntu, make sure you meet the prerequisites, then install Docker.

Prerequisites: OS requirements

To install Docker Engine, you need the 64-bit version of one of these Ubuntu versions:

- Ubuntu LTS
- Debian LTS
- Red Hat Enterprise Linux (RHEL) LTS
- Fedora LTS

Docker Engine is supported on x86_64 (or amd64), armhf, and arm64 architectures.

Uninstall old versions

Older versions of Docker were called **docker**, **docker.io**, or **docker-engine**. If these are installed, uninstall them:

```
$ sudo apt-get remove docker docker-engine docker.io containerd runc
```

It's OK if **apt-get** reports that none of these packages are installed.

The contents of **/var/lib/docker/**, including images, containers, volumes, and networks, are preserved. If you do not need to save your existing data, and want to start with a clean installation, refer to the uninstall Docker Engine section at the bottom of this page.

Supported storage drivers

Docker Engine on Ubuntu supports **overlay2**, **aufs** and **btrfs** storage drivers.

Docker Engine uses the **overlay2** storage driver by default. If you need to use **aufs** instead, you need to configure it manually.

Installation methods

You can install Docker Engine in different ways, depending on your needs:

- Most users set up Docker's repositories and install from them, for ease of installation and upgrade tasks. This is the recommended approach.
- Some users download the DEB package and install it manually and manage upgrades completely manually. This is useful in situations such as installing Docker on air-gapped systems with no access to the internet.
- In testing and development environments, some users choose to use automated convenience scripts to install Docker.

Install using the repository

Before you install Docker Engine for the first time on a new host machine, you need to set up the Docker repository. Afterward, you can install and update Docker from the repository.

SET UP THE REPOSITORY

1. Update the apt package index and install packages to allow apt to use a repository over HTTPS:

```
$ sudo apt-get update

$ sudo apt-get install \
  apt-transport-https \
  ca-certificates \
  curl \
  gnupg-agent \
  software-properties-common
```

Install using the repository

2. Add Docker's official GPG key:

```
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

Verify that you now have the key with the fingerprint **9DC8 5822 9FC7 DD38 854A E2D8 8D81 803C 0EBF CD88**, by searching for the last 8 characters of the fingerprint.

```
$ sudo apt-key fingerprint 0EBFCD88

pub   rsa4096 2017-02-22 [SCEA]
      9DC8 5822 9FC7 DD38 854A  E2D8 8D81 803C 0EBF CD88
uid           [ unknown] Docker Release (CE deb) <docker@docker.com>
sub   rsa4096 2017-02-22 [S]
```

Install using the repository

3. Use the following command to set up the stable repository. To add the nightly or test repository, add the word nightly or test (or both) after the word stable in the commands below.

```
$ sudo add-apt-repository \  
    "deb [arch=amd64] https://download.docker.com/linux/ubuntu \  
    $(lsb_release -cs) \  
    stable"
```

INSTALL DOCKER ENGINE

1. Update the apt package index, and install the latest version of Docker Engine and containerd, or go to the next step to install a specific version:

```
$ sudo apt-get update
$ sudo apt-get install docker-ce docker-ce-cli containerd.io
```

2. To install a specific version of Docker Engine, list the available versions in the repo, then select and install:

- a. List the versions available in your repo:

```
$ apt-cache madison docker-ce

docker-ce | 5:18.09.1~3-0~ubuntu-xenial | https://download.docker.com/linux/ubuntu | xenial/stable amd64 Packages
docker-ce | 5:18.09.0~3-0~ubuntu-xenial | https://download.docker.com/linux/ubuntu | xenial/stable amd64 Packages
docker-ce | 18.06.1~ce~3-0~ubuntu          | https://download.docker.com/linux/ubuntu | xenial/stable amd64 Packages
docker-ce | 18.06.0~ce~3-0~ubuntu          | https://download.docker.com/linux/ubuntu | xenial/stable amd64 Packages
...
```

INSTALL DOCKER ENGINE

b. Install a specific version using the version string from the second column, for example, 5:18.09.1~3-0~ubuntu-xenial.

```
$ sudo apt-get install docker-ce=<VERSION_STRING> docker-ce-cli=<VERSION_STRING> containerd.io
```

3. Verify that Docker Engine is installed correctly by running the hello-world image.

```
$ sudo docker run hello-world
```

This command downloads a test image and runs it in a container. When the container runs, it prints an informational message and exits.

UPGRADE DOCKER ENGINE

Docker Engine is installed and running. The docker group is created but no users are added to it. You need to use `sudo` to run Docker commands. Continue to Linux postinstall to allow non-privileged users to run Docker commands and for other optional configuration steps.

To upgrade Docker Engine, first run `sudo apt-get update`, then follow the installation instructions, choosing the new version you want to install.

Post-installation steps for Linux

This section contains optional procedures for configuring Linux hosts to work better with Docker

Manage Docker as a non-root user

The Docker daemon binds to a Unix socket instead of a TCP port. By default that Unix socket is owned by the user **root** and other users can only access it using **sudo**. The Docker daemon always runs as the root user.

If you don't want to preface the **docker** command with **sudo**, create a Unix group called **docker** and add users to it. When the Docker daemon starts, it creates a Unix socket accessible by members of the docker group.

Manage Docker as a non-root user

To create the docker group and add your user:

1. Create the docker group.

```
$ sudo groupadd docker
```

2. Add your user to the docker group.

```
$ sudo usermod -aG docker $USER
```

3. Log out and log back in so that your group membership is re-evaluated. If testing on a virtual machine, it may be necessary to restart the virtual machine for changes to take effect. On a desktop Linux environment such as X Windows, log out of your session completely and then log back in. On Linux, you can also run the following command to activate the changes to groups:

```
$ newgrp docker
```

Manage Docker as a non-root user

4. Verify that you can run docker commands without sudo.

```
$ docker run hello-world
```

This command downloads a test image and runs it in a container. When the container runs, it prints an informational message and exits.

If you initially ran Docker CLI commands using `sudo` before adding your user to the `docker` group, you may see the following error, which indicates that your `~/.docker/` directory was created with incorrect permissions due to the `sudo` commands.

```
WARNING: Error loading config file: /home/user/.docker/config.json -  
stat /home/user/.docker/config.json: permission denied
```

Manage Docker as a non-root user

To fix this problem, either remove the `~/.docker/` directory (it is recreated automatically, but any custom settings are lost), or change its ownership and permissions using the following commands:

```
$ sudo chown "$USER":"$USER" /home/"$USER"/.docker -R
$ sudo chmod g+rwX "$HOME/.docker" -R
```

Configure Docker to start on boot

Most current Linux distributions (RHEL, CentOS, Fedora, Ubuntu 16.04 and higher) use [systemd](#) to manage which services start when the system boots. Ubuntu 14.10 and below use [upstart](#).

systemd

```
$ sudo systemctl enable docker
```

To disable this behavior, use disable instead.

```
$ sudo systemctl disable docker
```

Configure Docker to start on boot

upstart

Docker is automatically configured to start on boot using upstart. To disable this behavior, use the following command:

```
$ echo manual | sudo tee /etc/init/docker.override
```

chkconfig

```
$ sudo chkconfig docker on
```

Use a different storage engine

For information about the different storage engines, see [Storage drivers](#). The default storage engine and the list of supported storage engines depend on your host's Linux distribution and available kernel drivers.

Configure default logging driver

Docker provides the capability to collect and view log data from all containers running on a host via a series of logging drivers. The default logging driver, **json-file**, writes log data to JSON-formatted files on the host filesystem. Over time, these log files expand in size, leading to potential exhaustion of disk resources. To alleviate such issues, either configure an alternative logging driver such as Splunk or Syslog, or set up log rotation for the default driver.

Configure where the Docker daemon listens for connections

By default, the Docker daemon listens for connections on a UNIX socket to accept requests from local clients. It is possible to allow Docker to accept requests from remote hosts by configuring it to listen on an IP address and port as well as the UNIX socket.

Configuring Docker to accept remote connections can be done with the `docker.service` systemd unit file for Linux distributions using systemd, such as recent versions of RedHat, CentOS, Ubuntu and SLES, or with the `daemon.json` file which is recommended for Linux distributions that do not use systemd.

Configuring remote access with systemd unit file

1. Use the command `sudo systemctl edit docker.service` to open an override file for `docker.service` in a text editor.
2. Add or modify the following lines, substituting your own values.

```
[Service]
ExecStart=
ExecStart=/usr/bin/dockerd -H fd:// -H tcp://127.0.0.1:2375
```

Configuring remote access with systemd unit file

3. Save the file.

4. Reload the systemctl configuration.

```
$ sudo systemctl daemon-reload
```

5. Restart Docker.

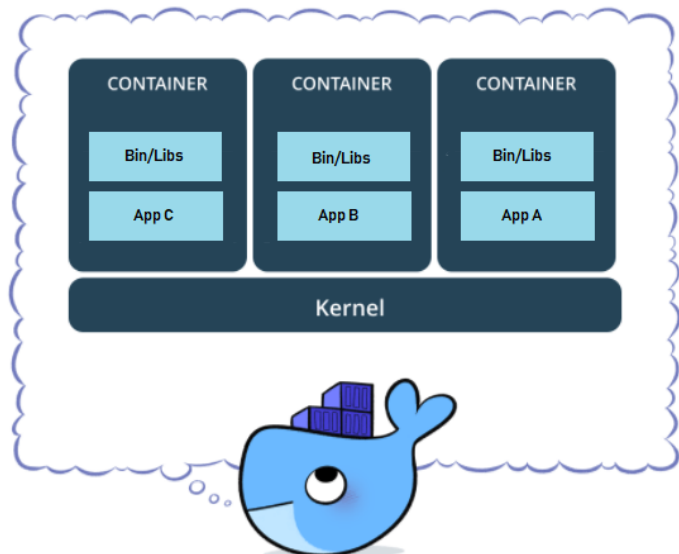
```
$ sudo systemctl restart docker.service
```

6. Check to see whether the change was honored by reviewing the output of netstat to confirm dockerd is listening on the configured port.

```
$ sudo netstat -lntp | grep dockerd
tcp        0      0 0.0.0.0:2375 0.0.0.0:*        LISTEN      3758/dockerd
```

Introduction to Docker Networking

- Docker networking allows containers to communicate with each other and with the external world.
- Provides isolation and network stack management using network drivers.



Docker Network Types

- **Bridge:** The default network type for containers, which enables communication between containers on the same host.
- **Host:** Removes network isolation between the container and the Docker host, allowing containers to use the host's networking directly.
- **Overlay:** Connects multiple Docker daemons together and enables swarm services to communicate.
- **Macvlan:** Allows assigning a MAC address to a container, making it appear as a physical device on your network.

Docker Bridge Network

The default network type when you run a container. The bridge network allows containers connected to the same bridge network to communicate with each other via IP addresses. Docker automatically assigns a private IP address to each container to prevent them from interfering with the host network.

Key Features:

- **Isolation:** Provides a high level of isolation as containers cannot communicate with the host by default, unless specific ports are mapped.
- **Ease of Setup:** Automatically configured by Docker, requiring no additional setup unless custom configurations are needed.
- **Use Cases:** Ideal for running multiple containers that need to communicate on the same Docker host, while keeping them isolated from the host network.
- **Commands:** Use `docker network create --driver bridge [network_name]` to create a custom bridge network and `docker run --network=[network_name] [other_options] [image]` to run a container on this network."

Docker Host Network

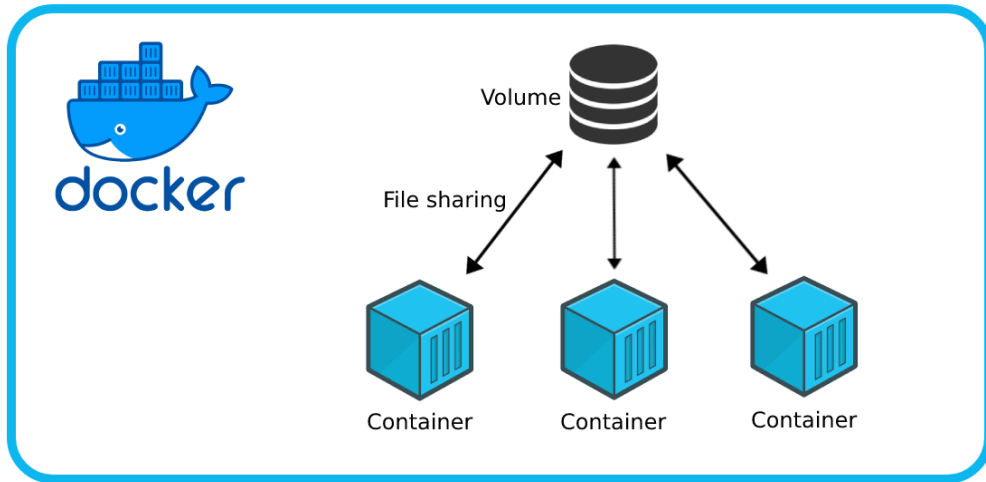
Bypasses Docker's managed networking infrastructure and allows containers to use the host's network directly. This means that every port opened in a container is accessible on the host's IP address.

Key Features:

- **Performance:** Offers the highest network performance as there is no network isolation between the host and the container, reducing overhead.
- **Direct Access:** Containers can directly use the host IP, making it easier to handle inbound traffic without port mapping.
- **Security Considerations:** Less secure than other network types because containers share the host's network namespace, potentially exposing the host system.
- **Use Cases:** Suitable for applications that require real-time or high-performance network communications where latency or network performance is critical.
- **Commands:** Run a container with the host network using `docker run --network host [other_options] [image]."`

Introduction to Docker Volumes

- Understand what Docker volumes are and why they are crucial for managing persistent data in Docker. Volumes are designed to survive container restarts, decoupling data lifecycle from container lifecycle, and providing an efficient way of handling data persistence.



Types of Docker Storage

- **Volumes:** Stored in a part of the host filesystem managed by Docker (`/var/lib/docker/volumes/` on Linux).
- **Bind mounts:** Can be stored anywhere on the host system and are often used for specific needs or legacy applications.
- **Tmpfs mounts:** Stored in the host system's memory only, ideal for sensitive information or non-persistent data."

Creating and Managing Volumes

- Volumes are a vital component of your containerized environment. Incredibly powerful and flexible, they provide persistent storage of data between containers and even after their removal.
- Creating and managing volumes is key to effective Docker usage. We'll explore strategies for creating and configuring volumes, as well as methods for efficiently managing them. Get ready to gain full control over your data!

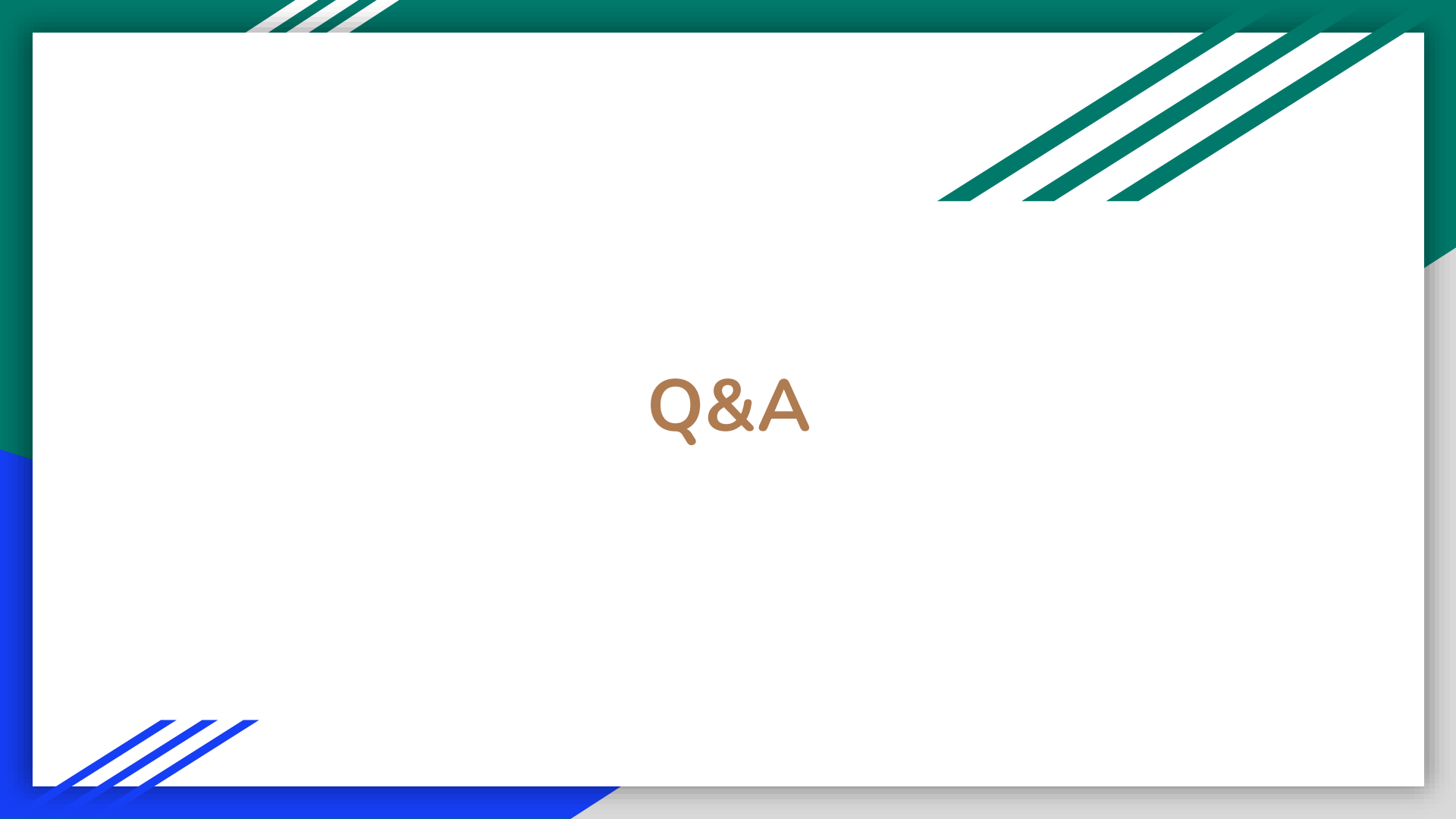
Key commands:

- `docker volume create`
- `docker volume ls`
- `docker volume rm`

Using Volumes with Docker Containers

- Volumes offer seamless integration with Docker containers, enabling you to effortlessly manage data persistence and sharing between containers. By leveraging volumes, you can ensure that your applications have access to essential data regardless of container lifecycle.
- Suppose you have a Dockerized database application, and you want to ensure that the database data persists even if the container is stopped or removed. You can achieve this using the following Docker command:

```
docker run -d --name my_database \  
-v /path/to/database/data:/var/lib/mysql \  
-e MYSQL_ROOT_PASSWORD=password \  
mysql:latest
```



Q&A